Python Cơ Bản Cho Người Mới Bắt Đầu

Tài liệu gốc kèm thực hành bổ sung · 29/10/2026

Mục lục

- Chương 1: Làm Quen Với Python: Cài đặt Python & IDE
- Chương 1: Làm Quen Với Python: Biến và kiểu dữ liệu
- Chương 1: Làm Quen Với Python: Toán tử trong Python
- Chương 2: Cấu Trúc Điều Khiển & Hàm: Câu lệnh if - else
- Chương 2: Cấu Trúc Điều Khiển & Hàm: Vòng lặp for, while
- Chương 2: Cấu Trúc Điều Khiển & Hàm: Định nghĩa và sử dụng hàm
- Chương 3: Lập Trình Hướng Đối Tượng Cơ Bản: Class và Object
- Chương 3: Lập Trình Hướng Đối Tượng Cơ Bản: Kế thừa (Inheritance)
- Chương 3: Lập Trình Hướng Đối Tượng Cơ Bản: Dự án nhỏ: Quản lý sinh viên

CHƯƠNG 1 · BÀI 1

1.1. Cài đặt Python & IDE

Tài liệu học tập Python Cơ Bản Cho Người Mới Bắt Đầu · Nội dung biên soạn trong dự án StudyOnline.

Mục tiêu bài học

- Kiểm tra được trình thông dịch Python đang sử dụng
- Tạo môi trường ảo và chạy chương trình từ terminal
- Phân biệt lỗi cú pháp, lỗi lúc chạy và lỗi logic

1. Python và môi trường làm việc

Mã nguồn Python là văn bản trong file .py. Trình thông dịch đọc và thực thi mã; IDE hỗ trợ soạn thảo, tìm lỗi và gỡ lỗi nhưng không thay thế trình thông dịch. Ví dụ của khóa dùng Python 3. Trên Windows có thể dùng trình khởi chạy py nếu lệnh python chưa được cấu hình.

```
py -3 --version
py -3 -m venv .venv
.\.venv\Scripts\python.exe --version
```

Môi trường ảo tách thư viện của một dự án khỏi dự án khác. Không cần kích hoạt môi trường nếu gọi trực tiếp python.exe bên trong .venv. Trong VS Code, chọn trình thông dịch của môi trường này để terminal và trình gỡ lỗi sử dụng cùng Python.

2. Chương trình đầu tiên

Tạo file hello.py trong thư mục dự án. Lưu file trước khi chạy; kiểm tra thư mục hiện tại nếu terminal báo không tìm thấy file.

```
name = "An"
print(f"Xin chào {name}")
print(2 + 3)
```

```
.\.venv\Scripts\python.exe hello.py
```

Kết quả là hai dòng: Xin chào An và 5. Nếu chương trình chạy trong IDE nhưng không chạy trong terminal, kiểm tra trình thông dịch và thư mục làm việc thay vì cài lại mọi thứ.

3. Đọc lỗi và gỡ lỗi

SyntaxError báo mã không đúng cú pháp, chẳng hạn thiếu dấu đóng ngoặc. NameError xuất hiện khi dùng tên chưa được định nghĩa. ValueError có thể xảy ra khi chuyển chuỗi không hợp lệ thành số. Lỗi logic là chương trình chạy nhưng kết quả không đúng yêu cầu.

Đọc traceback từ dòng cuối để biết loại lỗi, sau đó tìm tên file và số dòng thuộc mã của mình. Đặt breakpoint trước dòng sai, xem giá trị và kiểu của biến, chạy từng bước rồi so sánh với kết quả dự kiến. Không sửa nhiều chỗ cùng lúc vì khó biết thay đổi nào giải quyết lỗi.

Câu hỏi ôn tập và bài tập vận dụng

1. In tên, tuổi và tổng 12 + 8 từ một file Python.
2. Tạo lỗi bằng cách viết print(unknown_name), đọc traceback rồi sửa.
3. Viết phép tính diện tích hình chữ nhật; thử chiều dài 5, chiều rộng 3.

Đáp án và tiêu chí tự kiểm tra

1. Mã nằm trong file, chạy từ terminal thành công và phép cộng trả 20.
2. Lỗi NameError: khai báo unknown_name hoặc sửa thành tên biến đã tồn tại.
3. Diện tích là 15; dùng phép nhân, không dùng phép cộng. Giải thích được vì sao chương trình trả 8 vẫn chạy nhưng sai logic.

Tóm tắt

- IDE và trình thông dịch có vai trò khác nhau.
- Môi trường ảo giúp tách thư viện; dùng cùng trình thông dịch khi chạy và gỡ lỗi.
- Kiểm tra kết quả dự kiến và đọc traceback trước khi sửa mã.

Nguồn học tiếp

https://docs.python.org/3/tutorial/venv.html

Cài đặt Python & IDE — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: "Em đã cài Python nhưng VS Code báo không tìm thấy thư viện hoặc không chạy đúng file." Kiểm tra terminal đang đứng ở thư mục nào, tên file đã lưu và trình thông dịch được chọn. Môi trường ảo tách thư viện của dự án khỏi Python hệ thống; cài gói và chạy chương trình bằng cùng một interpreter.

```
import sys
print(sys.executable)
print("Xin chào StudyOnline")
```

Chạy `python -m venv .venv` trong thư mục dự án. Trên Windows có thể chạy trực tiếp `.venv\Scripts\python.exe hello.py` mà không cần thay đổi chính sách PowerShell. Không đặt tên file trùng thư viện như `json.py`.

Tự luyện: tạo chương trình in tên và một phép tính; chạy từ terminal và nút Run. **Hướng giải:** hai cách phải dùng cùng interpreter và cho cùng kết quả; chụp lỗi nếu khác nhau. Đọc dòng cuối traceback rồi tìm vị trí thuộc file của mình, không cài lại toàn bộ công cụ ngay.

Nguồn học tiếp

https://docs.python.org/3/tutorial/

CHƯƠNG 1 · BÀI 2

1.2. Biến và kiểu dữ liệu

Tài liệu học tập Python Cơ Bản Cho Người Mới Bắt Đầu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu biến là gì và cách đặt tên biến đúng quy tắc
> - Phân biệt được các kiểu dữ liệu cơ bản: int, float, str, bool
> - Biết kiểm tra và chuyển đổi kiểu dữ liệu

1. Biến trong Python

Biến là một cái tên dùng để lưu giá trị trong bộ nhớ. Trong Python không cần khai báo kiểu tường minh; kiểu được xác định lúc chạy, dựa vào giá trị được gán bằng dấu `=`.

```
ten = "An"
tuoi = 20
diem = 8.5
da_dang_ky = True
```

Quy tắc đặt tên: chỉ gồm chữ cái, chữ số và dấu gạch dưới; không bắt đầu bằng chữ số; phân biệt chữ hoa, chữ thường (`diem` khác `Diem`); không trùng từ khoá như `if`, `for`, `class`. Nên đặt tên viết thường, nối bằng dấu gạch dưới, ví dụ `so_luong_sinh_vien`.

2. Các kiểu dữ liệu cơ bản

- **int**: số nguyên, ví dụ `10`, `-3`.
- **float**: số thực, ví dụ `3.14`.
- **str**: chuỗi ký tự đặt trong dấu nháy đơn hoặc kép, ví dụ `"Xin chào"`.
- **bool**: chỉ có hai giá trị `True` và `False`.

Ngoài ra còn có `list` (danh sách có thể thay đổi), `tuple` (bất biến), `dict` (từ điển key-value) và `set` (tập hợp). Hàm `type(x)` trả về kiểu của giá trị x, ví dụ `type(3.14)` trả về `<class 'float'>`.

3. Chuyển đổi kiểu

Dùng các hàm `int()`, `float()`, `str()`, `bool()` để đổi kiểu. Hàm `input()` luôn trả về chuỗi nên cần ép kiểu khi tính toán.

```
tuoi = int(input("Nhập tuổi: "))
print("Năm sau bạn", tuoi + 1, "tuổi")
print(type(tuoi))   # <class 'int'>
```

Tóm tắt

- Biến lưu giá trị, kiểu được xác định theo giá trị gán.
- Bốn kiểu cơ bản: int, float, str, bool; dùng `type()` để kiểm tra.
- `input()` trả về chuỗi, cần `int()` hoặc `float()` khi tính toán.

Câu hỏi ôn tập và bài tập vận dụng

1. Khai báo biến lưu họ tên, tuổi, điểm trung bình của bạn rồi in ra kiểu của từng biến.
2. Nhập hai số từ bàn phím và in ra tổng của chúng.
3. Giải thích vì sao `"5" + "3"` cho kết quả khác `5 + 3`.

Biến và kiểu dữ liệu — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Hỏi thực tế: "Sao `print("2" + 3)` báo lỗi?" Trong Python, chuỗi và số nguyên không cộng trực tiếp. Muốn cộng số thì đổi chuỗi sang số; muốn ghép chữ thì đổi số sang chuỗi. `input()` luôn trả chuỗi, kể cả khi người dùng gõ 20.

```
assert int("2") + 3 == 5
assert "2" + str(3) == "23"

def next_age(raw):
    try:
        age = int(raw)
    except ValueError:
        return "Hãy nhập tuổi bằng chữ số"
    return age + 1 if age >= 0 else "Tuổi không được âm"

assert next_age("20") == 21
assert next_age("hai mươi") == "Hãy nhập tuổi bằng chữ số"
```

Tự luyện: thử chuỗi rỗng, khoảng trắng, số âm và `"2.5"`. **Hướng giải:** `int("2.5")` không hợp lệ; nếu bài toán chấp nhận số thực thì dùng float và kiểm tra miền giá trị. Không bắt mọi lỗi rồi trả 0 vì sẽ che dữ liệu sai.

Nguồn học tiếp

https://docs.python.org/3/tutorial/

CHƯƠNG 1 · BÀI 3

1.3. Toán tử trong Python

Tài liệu học tập Python Cơ Bản Cho Người Mới Bắt Đầu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Sử dụng được toán tử số học, so sánh và logic
> - Hiểu độ ưu tiên của các toán tử
> - Phân biệt toán tử gán `=` và toán tử so sánh `==`

1. Toán tử số học

Các toán tử số học gồm `+` (cộng), `-` (trừ), `*` (nhân), `/` (chia, luôn ra số thực), `//` (chia lấy phần nguyên), `%` (chia lấy dư) và `**` (luỹ thừa).

```
print(7 / 2)    # 3.5
print(7 // 2)   # 3
print(7 % 2)    # 1
print(2 ** 10)  # 1024
```

Toán tử `%` thường dùng để kiểm tra số chẵn, lẻ: `n % 2 == 0` là số chẵn.

2. Toán tử so sánh

Các toán tử so sánh trả về giá trị `True` hoặc `False`: `==` (bằng), `!=` (khác), `>`, `<`, `>=`, `<=`. Không được nhầm `==` (so sánh) với `=` (phép gán).

3. Toán tử logic

- `and`: đúng khi cả hai vế cùng đúng.
- `or`: đúng khi ít nhất một vế đúng.
- `not`: đảo ngược giá trị logic.

```
tuoi = 20
print(tuoi >= 18 and tuoi < 60)   # True
print(not (tuoi > 30))            # True
```

4. Toán tử gán rút gọn và độ ưu tiên

Có thể viết gọn `x += 5` thay cho `x = x + 5`; tương tự với `-=`, `*=`, `/=`. Độ ưu tiên từ cao xuống thấp: `**`, sau đó `* / // %`, rồi `+ -`, tiếp theo là so sánh, cuối cùng là `not`, `and`, `or`. Dùng dấu ngoặc để làm rõ ý.

Tóm tắt

- Số học: `+ - * / // % **`; so sánh cho kết quả True/False; logic dùng `and`, `or`, `not`.
- `==` là so sánh, `=` là gán.
- Dùng ngoặc để kiểm soát thứ tự tính.

Câu hỏi ôn tập và bài tập vận dụng

1. Nhập một số nguyên và in ra số đó chẵn hay lẻ (dùng `%`).
2. Tính diện tích và chu vi hình chữ nhật từ chiều dài, chiều rộng nhập vào.
3. Viết biểu thức logic kiểm tra một năm là năm nhuận.

Toán tử trong Python — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: nhập điểm hoặc tính tiền bị sai vì nhầm phép gán, so sánh và thứ tự tính. `=` gán giá trị; `==` kiểm tra bằng nhau; `/` chia thực, `//` lấy thương làm tròn xuống và `%` lấy phần dư. Với số âm, không coi `//` là cắt phần thập phân về 0.

```
assert 7 / 2 == 3.5
assert 7 // 2 == 3
assert -7 // 2 == -4
assert 7 % 2 == 1
score = 7
assert 0 <= score <= 10
assert (2 + 3) * 4 == 20
```

Tự luyện: tính số hộp đủ 6 sản phẩm và số sản phẩm dư cho 20 sản phẩm. **Hướng giải:** `20 // 6` bằng 3 hộp, `20 % 6` bằng 2 sản phẩm. Đổi mẫu chia sang 0 phải phát hiện dữ liệu không hợp lệ trước phép chia. Khi so số thực đo đạc, cân nhắc sai số thay vì dùng đẳng thức tuyệt đối tùy tiện.

Nguồn học tiếp

https://docs.python.org/3/tutorial/

CHƯƠNG 2 · BÀI 1

2.1. Câu lệnh if - else

Tài liệu học tập Python Cơ Bản Cho Người Mới Bắt Đầu · Nội dung biên soạn trong dự án StudyOnline.

Mục tiêu bài học

- Viết được câu lệnh rẽ nhánh if, elif, else
- Hiểu vai trò của thụt lề trong Python
- Kết hợp điều kiện bằng toán tử logic

1. Câu lệnh if

Câu lệnh `if` thực hiện một khối lệnh khi điều kiện đúng. Khối lệnh trong Python được xác định bằng thụt lề (thường 4 dấu cách), không dùng dấu ngoặc nhọn. Sau điều kiện phải có dấu hai chấm :.

```
diem = 7.5
if diem >= 5:
    print("Đạt")
```

2. if - else và elif

`else` xử lý trường hợp điều kiện sai. Khi có nhiều nhánh, dùng `elif`; Python kiểm tra lần lượt từ trên xuống và chỉ chạy nhánh đầu tiên đúng.

```
diem = 7.5
if diem >= 8:
    print("Giỏi")
elif diem >= 6.5:
    print("Khá")
elif diem >= 5:
    print("Trung bình")
else:
    print("Yếu")
```

3. Điều kiện phức hợp và if lồng nhau

Kết hợp nhiều điều kiện bằng `and`, `or`, `not`. Có thể đặt `if` bên trong `if` khác, nhưng nên hạn chế lồng quá sâu để code dễ đọc.

```
tuoi = 20
co_the = True
if tuoi >= 18 and co_the:
    print("Được đăng ký")
```

4. Biểu thức điều kiện một dòng

Python cho phép viết gọn: `ket_qua = "Đạt" if diem >= 5 else "Không đạt"`.

Tóm tắt

- `if`, `elif`, `else` dùng để rẽ nhánh; khối lệnh xác định bằng thụt lề.
- Chỉ nhánh đầu tiên có điều kiện đúng được thực hiện.
- Dùng `and`, `or`, `not` để ghép điều kiện.

Câu hỏi ôn tập và bài tập vận dụng

1. Nhập điểm và in ra xếp loại theo thang điểm Giỏi, Khá, Trung bình, Yếu.
2. Nhập ba số và tìm số lớn nhất bằng câu lệnh if.
3. Viết chương trình kiểm tra một năm có phải năm nhuận hay không.

Câu lệnh if - else — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Hỏi thực tế: "Em bị IndentationError: dòng print sau if viết thế nào?" Dòng thuộc khối if phải thụt vào. Dùng nhất quán bốn dấu cách, không trộn tab và khoảng trắng. Dấu hai chấm kết thúc dòng điều kiện.

```
if 5 > 2:
    print("đúng")

def classify(score):
    if not 0 <= score <= 10:
        raise ValueError("Điểm phải trong 0..10")
    if score >= 8:
        return "Tốt"
    elif score >= 5:
        return "Đạt"
    return "Cần ôn"

assert classify(5) == "Đạt"
assert classify(8) == "Tốt"
```

Tự luyện: thử -1, 0, 4.9, 5, 8 và 11. **Hướng giải:** điểm ngoài miền bị từ chối; điểm sát ngưỡng kiểm tra dấu `>=`. Kiểm tra ngưỡng cao trước để nhánh thấp không bắt mất trường hợp điểm cao. Muốn trợ giúp, gửi nguyên code có xuống dòng và thông báo lỗi đầy đủ.

Nguồn học tiếp

https://docs.python.org/3/tutorial/

CHƯƠNG 2 · BÀI 2

2.2. Vòng lặp for, while

Tài liệu học tập Python Cơ Bản Cho Người Mới Bắt Đầu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Sử dụng vòng lặp for với range và với danh sách
> - Sử dụng vòng lặp while khi chưa biết trước số lần lặp
> - Điều khiển vòng lặp bằng break và continue

1. Vòng lặp for

Vòng lặp `for` dùng để lặp qua từng phần tử của một dãy (list, tuple, str, range). Hàm `range(n)` sinh các số từ 0 đến n-1; `range(a, b)` từ a đến b-1; `range(a, b, bước)` cho phép đặt bước nhảy.

```
for i in range(1, 6):
    print(i)

for item in [10, 20, 30]:
    print(item)
```

2. Vòng lặp while

Vòng lặp `while` lặp trong khi điều kiện còn đúng. Cần đảm bảo điều kiện sẽ trở thành sai ở một thời điểm, nếu không chương trình chạy vô hạn.

```
tong = 0
n = 1
while n <= 100:
    tong += n
    n += 1
print("Tổng 1..100 =", tong)
```

3. break và continue

Lệnh `break` thoát khỏi vòng lặp ngay lập tức. Lệnh `continue` bỏ qua phần còn lại của vòng lặp hiện tại và sang vòng kế tiếp.

```
for i in range(1, 10):
    if i % 2 == 0:
        continue      # bỏ qua số chẵn
    if i > 7:
        break         # dừng khi i > 7
    print(i)          # in 1 3 5 7
```

4. Vòng lặp lồng nhau

Đặt một vòng lặp bên trong vòng lặp khác, ví dụ in bảng cửu chương: vòng ngoài chạy theo số nhân, vòng trong chạy từ 1 đến 10.

Tóm tắt

- `for` khi biết trước dãy cần lặp; `while` khi lặp theo điều kiện.
- `break` thoát vòng lặp, `continue` sang vòng kế tiếp.
- Cẩn thận vòng lặp vô hạn khi dùng `while`.

Câu hỏi ôn tập và bài tập vận dụng

1. In bảng cửu chương của một số nhập từ bàn phím.
2. Tính tổng các số chẵn từ 1 đến 100.
3. Dùng `while` để yêu cầu người dùng nhập mật khẩu cho đến khi đúng.

Vòng lặp for, while — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Hỏi thực tế: "In 1 đến 5 thì sao range lại ghi 6?" `range(a, b)` lấy a nhưng không lấy b. for duyệt các phần tử của một iterable; while lặp khi điều kiện còn đúng, cần bảo đảm có thay đổi dẫn tới kết thúc.

```
assert list(range(1, 6)) == [1, 2, 3, 4, 5]
total, n = 0, 1
while n <= 5:
    total += n
    n += 1
assert total == 15

values = [10, 20, 30]
assert values[0] == 10
assert values[2] == 30
```

Danh sách ba phần tử có chỉ số 0, 1, 2; `values[3]` gây IndexError. "Có ba số" không có nghĩa chỉ số lớn nhất là 3.

Tự luyện: cộng các số chẵn từ 1 tới 10, thử cả dãy rỗng. **Hướng giải:** duyệt `range(2, 11, 2)` cho tổng 30; dãy rỗng cho tổng 0. break thoát vòng lặp; continue bỏ phần còn lại của lượt hiện tại, nên không đặt việc cập nhật điều kiện while sau một continue có thể chạy mãi.

Nguồn học tiếp

https://docs.python.org/3/tutorial/

CHƯƠNG 2 · BÀI 3

2.3. Định nghĩa và sử dụng hàm

Tài liệu học tập Python Cơ Bản Cho Người Mới Bắt Đầu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Định nghĩa và gọi hàm bằng từ khoá def
> - Sử dụng tham số, tham số mặc định và giá trị trả về
> - Hiểu phạm vi của biến trong và ngoài hàm

1. Định nghĩa hàm

Hàm là một khối lệnh được đặt tên để tái sử dụng. Từ khoá `def` dùng để định nghĩa hàm; hàm trả về giá trị bằng `return`. Nếu không có `return`, hàm trả về `None`.

```
def cong(a, b):
    return a + b

print(cong(3, 4))   # 7
```

2. Tham số và đối số

Tham số là tên khai báo trong định nghĩa hàm, đối số là giá trị truyền vào khi gọi. Tham số có thể có giá trị mặc định, và có thể truyền theo tên.

```
def chao(ten, loi="Xin chào"):
    print(loi, ten)

chao("An")                  # Xin chào An
chao("Bình", loi="Chào bạn")  # Chào bạn Bình
```

3. Trả về nhiều giá trị

Hàm có thể trả về nhiều giá trị cùng lúc (dưới dạng tuple):

```
def min_max(ds):
    return min(ds), max(ds)
```

```
nho, lon = min_max([4, 9, 1, 7])
```

4. Phạm vi biến

Biến khai báo trong hàm là biến cục bộ, chỉ dùng được trong hàm đó. Biến khai báo ngoài hàm là biến toàn cục. Nên truyền dữ liệu vào hàm bằng tham số và lấy kết quả bằng `return` thay vì dùng biến toàn cục.

Tóm tắt

- `def` định nghĩa hàm, `return` trả về kết quả (mặc định `None`).
- Tham số có thể có giá trị mặc định; có thể trả về nhiều giá trị.
- Biến trong hàm là biến cục bộ.

Câu hỏi ôn tập và bài tập vận dụng

1. Viết hàm kiểm tra một số có phải số nguyên tố hay không.
2. Viết hàm tính giai thừa của n.
3. Viết hàm nhận danh sách điểm và trả về điểm trung bình.

Định nghĩa và sử dụng hàm — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Hỏi thực tế: "return và print có giống nhau không?" print hiển thị; return kết thúc hàm và chuyển kết quả cho nơi gọi. return là từ khóa, không phải tên một hàm. Hàm không return giá trị sẽ trả None.

```
def add(a, b):
    return a + b

assert add(2, 3) == 5
result = add(2, 3)
print(result)

def append_item(item, items=None):
    if items is None:
        items = []
    return items + [item]

assert append_item("An") == ["An"]
assert append_item("Bình") == ["Bình"]
```

Tự luyện: viết hàm tính trung bình không in bên trong; quy định rõ danh sách rỗng. **Hướng giải:** trả số hoặc báo ValueError theo hợp đồng, rồi assert giá trị trả về. Đừng dùng danh sách mutable làm đối số mặc định nếu các lần gọi cần độc lập.

Nguồn học tiếp

https://docs.python.org/3/tutorial/

CHƯƠNG 3 · BÀI 1

3.1. Class và Object

Tài liệu học tập Python Cơ Bản Cho Người Mới Bắt Đầu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu khái niệm lớp (class) và đối tượng (object)
> - Định nghĩa lớp với thuộc tính và phương thức
> - Sử dụng phương thức khởi tạo `__init__` và tham số `self`

1. Lớp và đối tượng

Lập trình hướng đối tượng (OOP) mô hình hoá chương trình thành các đối tượng. Lớp (class) là bản thiết kế mô tả thuộc tính và hành vi; đối tượng (object) là một thể hiện cụ thể của lớp. Từ khoá `class` dùng để định nghĩa lớp.

2. Phương thức khởi tạo và self

Phương thức khởi tạo là `__init__`, được gọi tự động khi tạo đối tượng. Tham số đầu tiên của mọi phương thức thường là `self`, tham chiếu tới đối tượng hiện tại.

```
class SinhVien:
    def __init__(self, ten, diem):
        self.ten = ten
        self.diem = diem

    def xep_loai(self):
        if self.diem >= 8:
            return "Giỏi"
        return "Khá" if self.diem >= 6.5 else "Trung bình"

sv = SinhVien("An", 8.5)
print(sv.ten, sv.xep_loai())   # An Giỏi
```

3. Thuộc tính lớp và phương thức đặc biệt

Thuộc tính khai báo ngay trong thân lớp (ngoài `__init__`) là thuộc tính lớp, dùng chung cho mọi đối tượng. Phương thức `__str__` quy định chuỗi hiển thị khi in đối tượng bằng `print()`.

```
class SinhVien:
    truong = "ĐH Công nghệ"      # thuộc tính lớp
    def __str__(self):
        return f"{self.ten} ({self.diem})"
```

Tóm tắt

- Lớp là bản thiết kế, đối tượng là thể hiện của lớp.
- `__init__` khởi tạo đối tượng; `self` tham chiếu tới chính đối tượng.
- `__str__` tuỳ biến cách in đối tượng.

Câu hỏi ôn tập và bài tập vận dụng

1. Viết lớp HinhChuNhat có phương thức tính diện tích và chu vi.
2. Viết lớp TaiKhoan có phương thức nạp tiền, rút tiền (không cho rút quá số dư).
3. Tạo 3 đối tượng SinhVien và in danh sách ra màn hình.

Class và Object — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: nhiều học viên cùng tên nhưng cần quản lý riêng. Class mô tả cấu trúc và hành vi; object là một đối tượng cụ thể. Thuộc tính thay đổi theo từng học viên phải đặt trên instance, tránh một danh sách dùng chung ngoài ý muốn.

```
class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name
        self.scores = []

    def add_score(self, score):
        if not 0 <= score <= 10:
            raise ValueError("Điểm không hợp lệ")
        self.scores.append(score)

a, b = Student(1, "An"), Student(2, "An")
a.add_score(8)
assert a.scores == [8] and b.scores == []
```

Tự luyện: thêm phương thức tính trung bình và trường hợp chưa có điểm. **Hướng giải:** xử lý danh sách rỗng rõ ràng; không chia 0 hoặc tự ghi điểm 0 cho người chưa làm bài. Tên trùng không đồng nghĩa cùng một object hoặc cùng mã học viên.

Nguồn học tiếp

https://docs.python.org/3/tutorial/

CHƯƠNG 3 · BÀI 2

3.2. Kế thừa (Inheritance)

Tài liệu học tập Python Cơ Bản Cho Người Mới Bắt Đầu · Nội dung biên soạn trong dự án StudyOnline.

Mục tiêu bài học

- Hiểu tính kế thừa và lợi ích của nó
- Viết lớp con kế thừa lớp cha, gọi phương thức lớp cha bằng `super()`
- Ghi đè (override) phương thức của lớp cha

1. Kế thừa là gì

Kế thừa (inheritance) cho phép lớp con dùng lại thuộc tính và phương thức của lớp cha, giúp tránh lặp code. Lớp con được khai báo bằng cách đặt tên lớp cha trong dấu ngoặc.

```
class Nguoi:
    def __init__(self, ten):
        self.ten = ten
    def gioi_thieu(self):
        print("Tôi là", self.ten)

class SinhVien(Nguoi):
    def __init__(self, ten, mssv):
        super().__init__(ten)
        self.mssv = mssv
```

2. Hàm super()

Hàm `super()` trả về đối tượng đại diện cho lớp cha, dùng để gọi phương thức của lớp cha, thường là `__init__`, để lớp con không phải viết lại phần khởi tạo chung.

3. Ghi đè phương thức

Lớp con có thể định nghĩa lại một phương thức của lớp cha với cùng tên để thay đổi hành vi. Đây là cơ sở của đa hình: cùng một lời gọi phương thức nhưng mỗi đối tượng xử lý theo cách riêng.

```
class SinhVien(Nguoi):
    def gioi_thieu(self):
```

```
        super().gioi_thieu()
        print("MSSV:", self.mssv)
```

4. Kiểm tra quan hệ kế thừa

`isinstance(sv, Nguoi)` cho biết đối tượng có thuộc lớp (hoặc lớp cha của lớp) đó hay không; `issubclass(SinhVien, Nguoi)` kiểm tra quan hệ giữa hai lớp. Python hỗ trợ đa kế thừa, tức một lớp có thể kế thừa nhiều lớp cha.

Tóm tắt

- Lớp con kế thừa thuộc tính, phương thức của lớp cha.
- `super()` gọi lớp cha; ghi đè phương thức để thay đổi hành vi.
- `isinstance` và `issubclass` kiểm tra quan hệ kế thừa.

Câu hỏi ôn tập và bài tập vận dụng

1. Viết lớp Dong_vat và hai lớp con Cho, Meo cùng ghi đè phương thức `keu()`.
2. Viết lớp NhanVien kế thừa Nguoi, bổ sung thuộc tính lương và phương thức tính lương.
3. Giải thích sự khác nhau giữa ghi đè và gọi `super()` trong một phương thức.

Kế thừa (Inheritance) — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: học viên thường và học viên học bổng có chung thông tin nhưng cách tính học phí khác. Kế thừa phù hợp quan hệ "là một"; thành phần được chứa bên trong nên cân nhắc composition.

```
class Learner:
    def __init__(self, name):
        self.name = name
    def tuition(self):
        return 100

class ScholarshipLearner(Learner):
    def tuition(self):
        return super().tuition() * 0.5

assert ScholarshipLearner("An").tuition() == 50
assert ScholarshipLearner("An").name == "An"
```

Tự luyện: cho hai loại đối tượng vào cùng một danh sách và gọi tuition. **Hướng giải:** phương thức ghi đè được chọn theo đối tượng; không cần chuỗi if kiểm tra mọi tên lớp. Lớp học "có nhiều học viên" không nên kế thừa Student chỉ để dùng lại thuộc tính.

Nguồn học tiếp

https://docs.python.org/3/tutorial/

CHƯƠNG 3 · BÀI 3

3.3. Dự án nhỏ: Quản lý sinh viên

Tài liệu học tập Python Cơ Bản Cho Người Mới Bắt Đầu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Áp dụng OOP để xây dựng chương trình quản lý sinh viên
> - Kết hợp lớp, danh sách, hàm và vòng lặp trong một dự án nhỏ
> - Rèn kỹ năng chia chương trình thành các phần nhỏ

1. Yêu cầu bài toán

Xây dựng chương trình console quản lý danh sách sinh viên với các chức năng: thêm sinh viên, hiển thị danh sách, tìm theo tên, xoá sinh viên và thống kê điểm trung bình.

2. Thiết kế

- Lớp `SinhVien` lưu mã, tên, điểm và có phương thức `xep_loai()`.
- Lớp `QuanLySinhVien` chứa danh sách (list) các sinh viên và các phương thức xử lý: `them`, `hien_thi`, `tim_theo_ten`, `xoa`, `diem_trung_binh`.
- Hàm `main()` hiển thị menu, nhận lựa chọn của người dùng bằng vòng lặp `while`.

3. Cài đặt mẫu

```
class SinhVien:
    def __init__(self, ma, ten, diem):
        self.ma, self.ten, self.diem = ma, ten, diem

    def xep_loai(self):
        return "Giỏi" if self.diem >= 8 else "Khá" if self.diem >= 6.5 else "Trung
bình"

class QuanLySinhVien:
    def __init__(self):
        self.ds = []

    def them(self, sv):
        self.ds.append(sv)
```

```
    def tim_theo_ten(self, tu_khoa):
        return [sv for sv in self.ds if tu_khoa.lower() in sv.ten.lower()]

    def diem_trung_binh(self):
        return sum(sv.diem for sv in self.ds) / len(self.ds) if self.ds else 0
```

4. Gợi ý mở rộng

Lưu danh sách ra file (văn bản hoặc JSON) để dữ liệu không mất khi tắt chương trình; kiểm tra dữ liệu nhập (điểm từ 0 đến 10, mã không trùng); sắp xếp danh sách theo điểm.

Tóm tắt

- Chia bài toán thành các lớp có trách nhiệm rõ ràng.
- Dùng list để lưu tập đối tượng, duyệt bằng vòng lặp hoặc list comprehension.
- Menu bằng `while` giúp chương trình chạy liên tục đến khi người dùng thoát.

Câu hỏi ôn tập và bài tập vận dụng

1. Hoàn thiện chương trình với đầy đủ menu và các chức năng ở trên.
2. Thêm chức năng sắp xếp danh sách theo điểm giảm dần.
3. Thêm chức năng ghi và đọc danh sách sinh viên từ file JSON.

CHƯƠNG HỌC THÊM · BÀI TỔNG HỢP

Học thêm.Tổng hợp. Học thêm, bài tập và hướng dẫn giải

Tài liệu học tập Python Cơ Bản Cho Người Mới Bắt Đầu · Nội dung biên soạn trong dự án StudyOnline.

Python — Sổ tay học thêm và thực hành tổng hợp

Tài liệu tự biên soạn bổ sung cho chín bài Python của khóa. Đọc sau phần chính; không cần học thuộc mọi ví dụ. Mục tiêu là giải thích được kết quả và tự kiểm tra chương trình.

Mục tiêu học thêm

- Phân biệt phép gán, sao chép và thay đổi đối tượng
- Kiểm tra dữ liệu đầu vào và các giá trị biên
- Tổ chức chương trình quản lý sinh viên bằng hàm và lớp

1. Biến tham chiếu tới đối tượng

Phép gán không luôn tạo bản sao. Với danh sách, hai tên có thể cùng trỏ tới một đối tượng:

```
a = [7, 8]
b = a
b.append(9)
print(a)  # [7, 8, 9]
c = a.copy()
c.append(10)
print(a)  # [7, 8, 9]
```

copy() là sao chép nông: danh sách ngoài mới nhưng đối tượng lồng bên trong vẫn có thể dùng chung. Với danh sách số như ví dụ, sao chép nông đủ để append không ảnh hưởng danh sách gốc. Đây là lỗi thường gặp khi tạo nhiều danh sách điểm từ cùng một mẫu.

2. Điều kiện, vòng lặp và giá trị biên

Với xếp loại điểm, kiểm tra đầu vào trước khi phân loại. Đặt điều kiện điểm cao trước khi dùng elif theo ngưỡng giảm dần để không nuốt các trường hợp sau.

```
def classify(score):
    if not 0 <= score <= 10:
```

```
        raise ValueError("Điểm phải từ 0 đến 10")
    if score >= 8.5:
        return "Tốt"
    if score >= 7:
        return "Khá"
    if score >= 5:
        return "Đạt"
    return "Cần ôn"

assert classify(5) == "Đạt"
assert classify(8.5) == "Tốt"
```

range(1, 4) tạo 1, 2, 3, không có 4. Khi kiểm tra vòng lặp, thử danh sách rỗng, một phần tử và nhiều phần tử. Với while, xác định biến nào làm điều kiện thay đổi; nếu không thay đổi, vòng lặp có thể không kết thúc.

3. Hàm thuần và lỗi đối số mặc định

Hàm tính toán nên trả kết quả, còn nhập/xuất thực hiện ở lớp giao diện. Nhờ vậy có thể kiểm tra bằng assert mà không phải nhập tay. Tránh đối số mặc định là danh sách nếu mỗi lần gọi cần một danh sách mới.

```
def add_student(name, students=None):
    if students is None:
        students = []
    students.append(name)
    return students

assert add_student("An") == ["An"]
assert add_student("Bình") == ["Bình"]
```

Nếu dùng students=[] trong định nghĩa hàm, danh sách được tạo một lần khi định nghĩa, nên các lần gọi có thể giữ dữ liệu cũ. None giúp tạo danh sách mới cho mỗi lần không truyền đối số.

4. Lớp, kế thừa và thiết kế dự án

Student nên chịu trách nhiệm về một sinh viên; danh sách và thao tác tìm kiếm nên thuộc lớp quản lý hoặc hàm quản lý. Dùng kế thừa khi có quan hệ “là một”, không dùng chỉ để tránh viết vài dòng giống nhau.

```
class Student:
    def __init__(self, student_id, name, scores):
        if not scores or any(not 0 <= s <= 10 for s in scores):
            raise ValueError("Cần điểm hợp lệ")
        self.student_id = student_id
        self.name = name.strip()
        self.scores = list(scores)

    def average(self):
        return sum(self.scores) / len(self.scores)

s = Student("SV01", "An", [8, 6])
assert s.average() == 7
```

Danh sách điểm được sao chép để append vào danh sách bên ngoài không tự thay đổi điểm sinh viên. Trong ứng dụng hoàn chỉnh còn cần kiểm tra tên rỗng và mã trùng khi thêm vào hệ thống.

Câu hỏi ôn tập và bài tập vận dụng

1. Dễ: dự đoán a sau khi b = a rồi b.append(5); giải thích khác biệt với b = a.copy().
2. Trung bình: viết hàm nhận danh sách điểm, trả trung bình hoặc None nếu rỗng; không làm tròn trong quá trình cộng.
3. Khó: thiết kế quản lý sinh viên với thêm, cập nhật, tìm theo mã và xếp loại. Khi mã trùng hoặc điểm ngoài khoảng, không được thay đổi dữ liệu cũ.

Gợi ý lời giải và tiêu chí đánh giá

1. a và b dùng chung danh sách nên cùng nhìn thấy phần tử mới. Với copy, append chỉ thay danh sách b.
2. Kiểm tra if not scores trước khi chia; dùng sum(scores) / len(scores). Thử [], [0], [10], [6, 8].
3. Lưu sinh viên trong dict theo mã để phát hiện trùng. Kiểm tra dữ liệu trước khi ghi; tách validate, add và update. Kiểm thử mã không tồn tại, tên trắng, điểm -1, điểm 11 và sửa sinh viên đang có. Bài đạt khi có thể giải thích cấu trúc dữ liệu và chứng minh cập nhật lỗi không làm mất bản ghi.

Học tiếp và tự đánh giá

Tự viết thêm chức năng lưu JSON, xử lý lỗi file và đọc lại dữ liệu. So sánh dữ liệu trước và sau một vòng lưu/đọc. Không dùng eval để đọc dữ liệu người dùng. Trước khi chuyển sang framework, cần chạy được chương trình bằng terminal và có các kiểm tra giá trị biên.

Nguồn đối chiếu: https://docs.python.org/3/tutorial/datastructures.html ; https://docs.python.org/3/tutorial/controlflow.html ; https://docs.python.org/3/tutorial/classes.html

Dự án nhỏ: Quản lý sinh viên — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Yêu cầu thực hành: lưu mã, tên và danh sách điểm; từ chối mã trùng, tên rỗng và điểm ngoài 0..10. Chia tách nhập liệu, xử lý và xuất kết quả. Không sửa database LMS thật để làm bài tập.

```
def add_student(registry, student_id, name):
    if student_id in registry:
        raise ValueError("Mã đã tồn tại")
    if not name.strip():
        raise ValueError("Tên rỗng")
    registry[student_id] = {"name": name.strip(), "scores": []}

students = {}
add_student(students, "SV01", " An ")
assert students["SV01"]["name"] == "An"
```

Tự luyện: thêm, tìm, sửa tên, tính trung bình và lưu JSON vào file riêng. **Hướng giải:** kiểm tra trước khi ghi; lưu được rồi đọc lại để so dữ liệu. Báo cáo cần có ca mã trùng, không tìm thấy, chưa có điểm, file không tồn tại và JSON lỗi. Gợi ý học 30 phút: 5 phút ôn, 15 phút viết một chức năng, 10 phút tự kiểm tra; đây là gợi ý tự học, không phải lịch thi chính thức.

Nguồn học tiếp

https://docs.python.org/3/tutorial/